GRAPHES SANS CIRCUIT

F. M.

2022/2023

SOMMAIRE

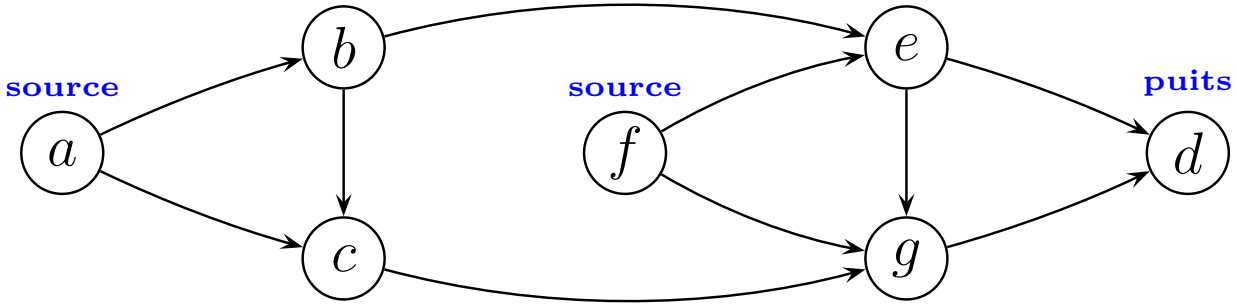
1. PARTICULARITÉS
2. ALGORITHME DE DÉCISION
3. ORDRE TOPOLOGIQUE
4. NOTION D'ORDONNANCEMENT
5. ALGORITHME DE TRI TOPOLOGIQUE
6. COMPLÉMENTS

1. PARTICULARITÉS

Dans ce chapitre, $G = (S, A)$ désigne un graphe **orienté**, **simple** et **connexe**.

Propriété 1.1

- Si G est sans source alors G contient au moins un circuit.
  Contraposée : si G est sans circuit alors G possède une source.
- Si G est sans puits alors G contient au moins un circuit.
  Contraposée : si G est sans circuit alors G possède un puits.



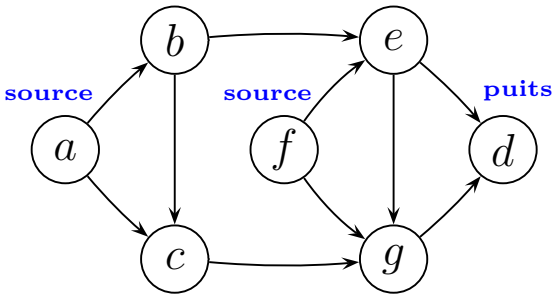
DAG

Dans la suite, on utilisera l'acronyme **DAG** (Directed Acyclic Graph) pour désigner un graphe orienté sans circuit.

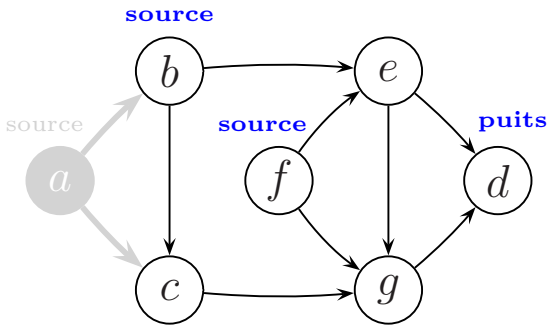
Propriété 1.2

- Tout chemin d'un DAG est élémentaire.
- Pour tout sommet x d'un DAG il existe un chemin élémentaire allant d'une source du DAG vers x.
- Si s est l'unique source d'un DAG alors s est une racine du graphe.
- Tout sous-graphe d'un DAG est un DAG. Par exemple, si on supprime un sommet x d'un DAG G alors $G - x$ est toujours un DAG.

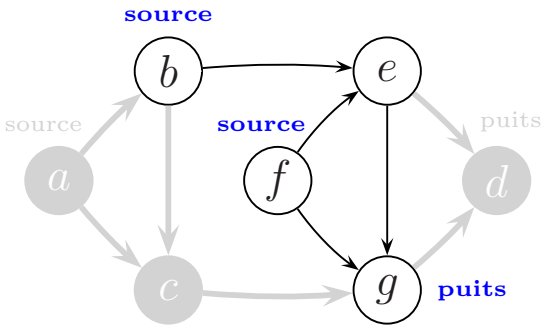
Illustration



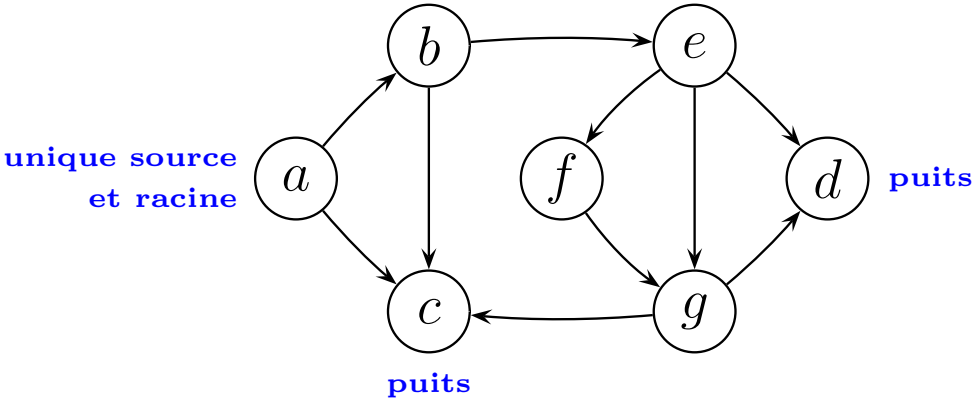
G est un DAG



$G - a$ est un DAG



$G - \{a, c, d\}$ est un DAG



l'unique source de ce graphe est aussi une racine

Que peut-on dire à propos d'un DAG admettant un unique puits ?

Propriété 1.3

Un graphe orienté G est un DAG si et seulement si il vérifie les deux conditions suivantes :

(i) G possède au moins une source.

(ii) Pour chaque source s de G, $G - s$ est un DAG.

On obtient une propriété équivalente en remplaçant le mot source par le mot puits.

($\Rightarrow$) Soit G un DAG. Alors G possède au moins une source (propriété 1.1) et tout sous-graphe de G est un DAG (propriété 1.2).

($\Leftarrow$) Soit s une source d'un graphe G (c.-à-d. $\delta^-(s) = 0$) telle que G-s soit un DAG. Supposons que G contienne un circuit. Puisque $G - s$ ne contient aucun circuit, la seule possibilité pour que G admette un circuit est que ce circuit passe par s. Le sommet s aurait alors un prédécesseur dans G et $\delta^-(s) \geq 1$ (impossible). L'hypothèse que G contienne un circuit est donc fausse.

Algorithme de décision

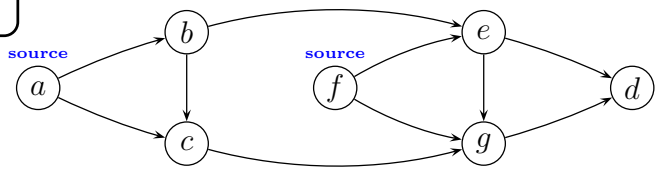
Cet algorithme est fondé sur les propriétés précédentes. Il permet de décider si, oui ou non, un graphe est sans circuit.

Donnée : G, un graphe orienté
Résultat : la réponse à la question, G est-il un DAG ?

tant que G *contient une source* s **faire**
  | $G \leftarrow G - s$
fin tant que
si G *est vide* **alors**
  | ecrire ("oui : le graphe est un DAG")
sinon
  | ecrire ("non : le graphe contient des circuits")
fin si

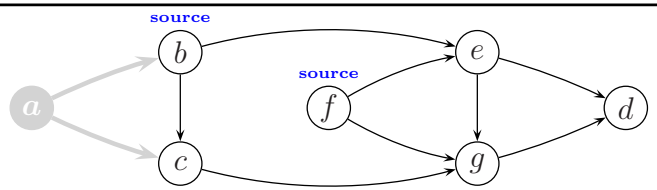
Illustration

init. [0]



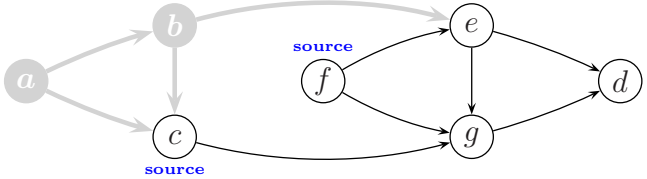
cand. : $\{a, f\}$

ité. [1]



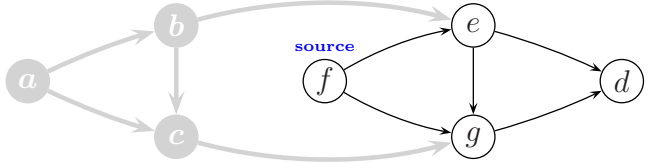
suppr. a $\Rightarrow$ cand. : $\{b, f\}$

ité. [2]



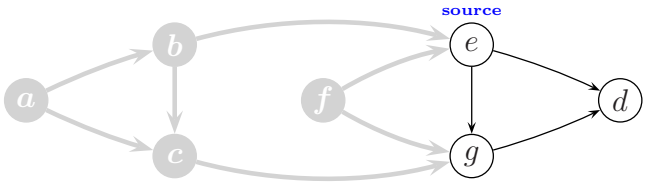
suppr. b $\Rightarrow$ cand. : $\{c, f\}$

ité. [3]



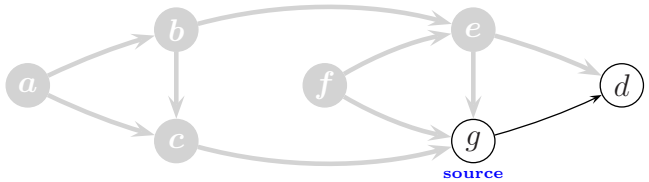
suppr. c $\Rightarrow$ cand. : $\{f\}$

ité. [4]



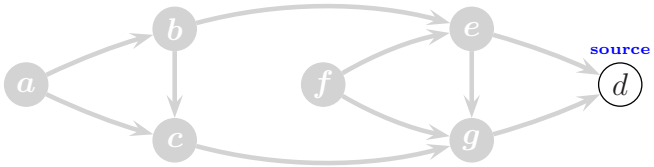
suppr. f $\Rightarrow$ cand. : $\{e\}$

ité. [5]



suppr. e $\Rightarrow$ cand. : $\{g\}$

ité. [6]



suppr. g $\Rightarrow$ cand. : $\{d\}$

ité. [7]

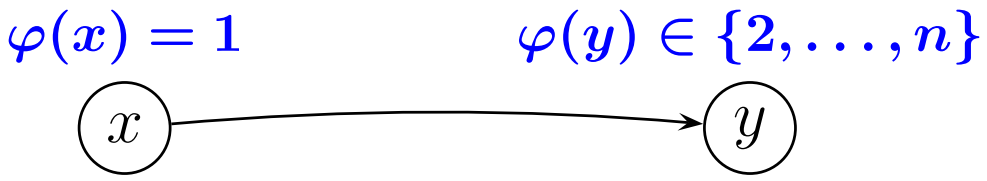
suppr. d, graphe vide $\Rightarrow$ pas de circuit

Définition 3.1

Soit $G = (S, A)$ un graphe d'ordre n. Un **ordre topologique** pour G est une bijection $\varphi : S \mapsto \{1, \ldots, n\}$ vérifiant :

$$\forall (x, y) \in A, \quad \varphi(x) < \varphi(y)$$

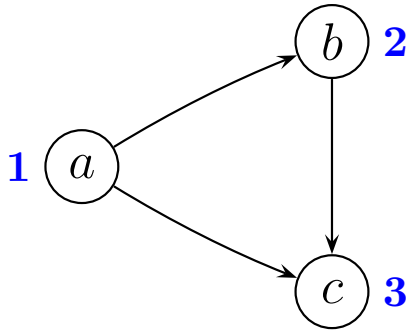
Le nombre $\varphi(x)$ est appelé **numéro topologique** du sommet x.



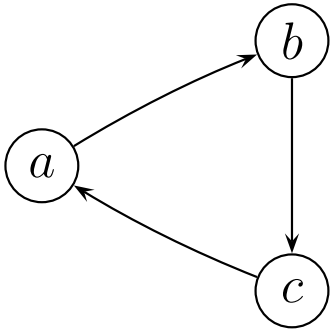
Liste associée à un ordre topologique

Soient G un graphe et L la liste des sommets de G triés selon un ordre topologique φ : $\varphi(x)$ est le rang de x dans L. Par extension, on dira que la liste L est un ordre topologique pour G.

Exemples



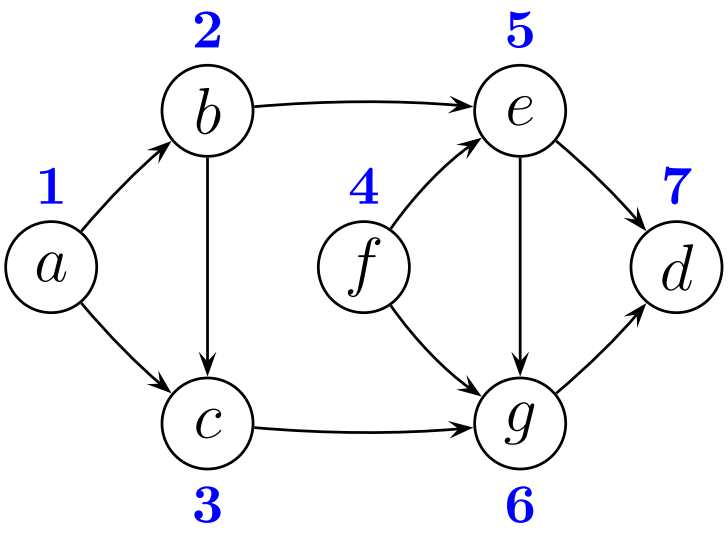
la liste $L = (a, b, c)$ est un ordre topologique pour ce graphe

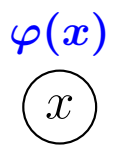


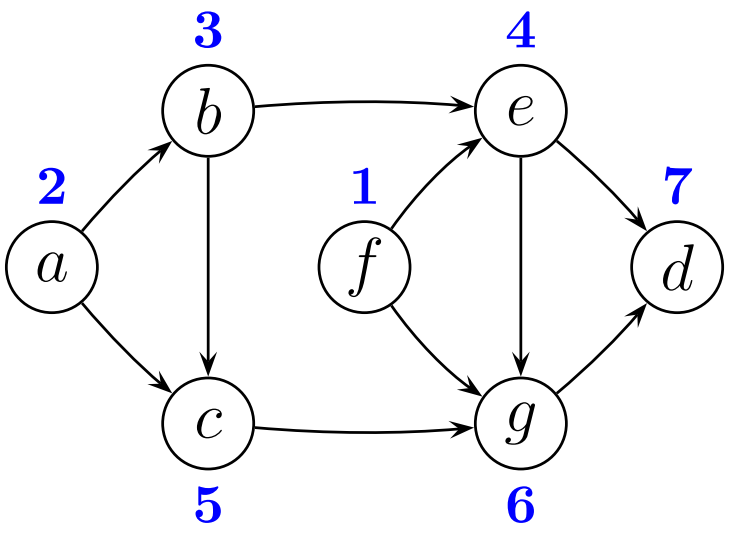
circuit $\Rightarrow$ aucun ordre topologique

Exemples (suite)

$$\varphi : \{a, b, c, d, e, f, g\} \mapsto \{1, 2, 3, 4, 5, 6, 7\}$$







$L = (a, b, c, f, e, g, d)$

$L = (f, a, b, e, c, g, d)$

Un DAG admet au moins un ordre topologique (il n'y a pas unicité)

Constatations

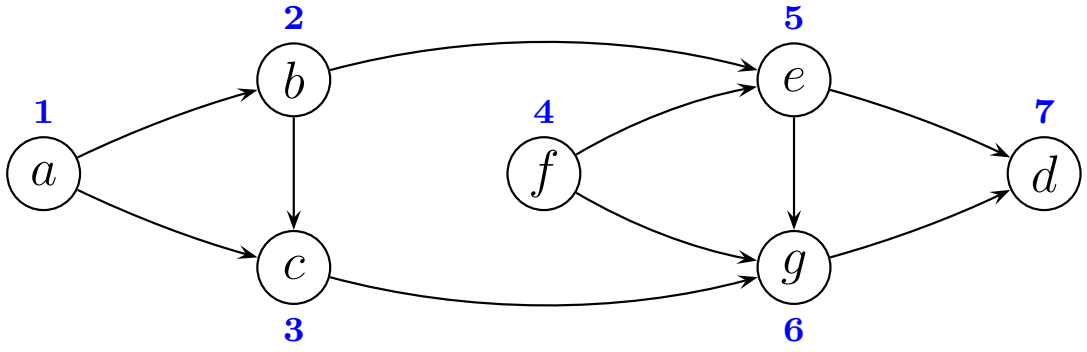
Soient G un DAG d'ordre n et φ un ordre topologique pour G.

- $\varphi(x) = 1 \Rightarrow x$ est une source de G. La réciproque est fausse puisqu'un graphe peut contenir plusieurs sources.
- $\varphi(x) = n \Rightarrow x$ est un puits de G. La réciproque est fausse puisqu'un graphe peut contenir plusieurs puits.
- Si $(s_1, s_2, \ldots, s_k)$ est un chemin de G alors :

$$\varphi(s_1) < \varphi(s_2) < \cdots < \varphi(s_k)$$

  Rappelons que tout chemin de G est élémentaire.

Illustration



$$\varphi(a) = 1 \Rightarrow a \text{ est une source}$$

$$f \text{ est une source} \not\Rightarrow \varphi(f) = 1$$

$$\varphi(d) = 7 \Rightarrow d \text{ est un puits}$$

$$(a, b, e, d) \text{ est un chemin} \Rightarrow \varphi(a) < \varphi(b) < \varphi(e) < \varphi(d)$$

Propriété 3.1

Soit G un graphe orienté d'ordre n. Alors : G est sans circuit si et seulement si il existe un ordre topologique pour G.

($\Rightarrow$) Soit φ un ordre topologique pour G. Hypothèse : G contient un circuit $(s_1, s_2, \ldots, s_k)$ où $s_1 = s_k$. Alors l'ordre topologique implique que $\varphi(s_1) < \varphi(s_2) < \cdots < \varphi(s_k) = \varphi(s_1)$, ce qui aboutit à la contradiction $\varphi(s_1) < \varphi(s_1)$. L'hypothèse est donc fausse.
($\Leftarrow$) On effectue une démonstration par récurrence sur n. Si $n = 1$, l'implication est vraie. Supposons que tout DAG d'ordre n possède un ordre topologique. Soit G' un DAG d'ordre $n+1$. Alors G' contient un puits p (propriété 1.2) et le sous-graphe $G = G' - p$ est un DAG d'ordre n (propriété 1.3). Par hypothèse de récurrence, G possède un ordre topologique φ et, en posant $\varphi(p) = n+1$, on obtient alors un ordre topologique pour G'. En effet, pour chaque prédécesseur x de p, l'arc (x, p) vérifie $\varphi(x) < \varphi(p)$.

3. ORDRE TOPOLOGIQUE

Diagramme d'un ordre topologique

Soit G un DAG et φ un ordre topologique pour G. On place les sommets de G le long d'un axe horizontal selon l'ordre topologique φ : le sommet de numéro topologique 1 d'abord, puis celui de numéro topologique 2, etc. Une fois les sommets ainsi alignés, on ajoute les arcs de G.

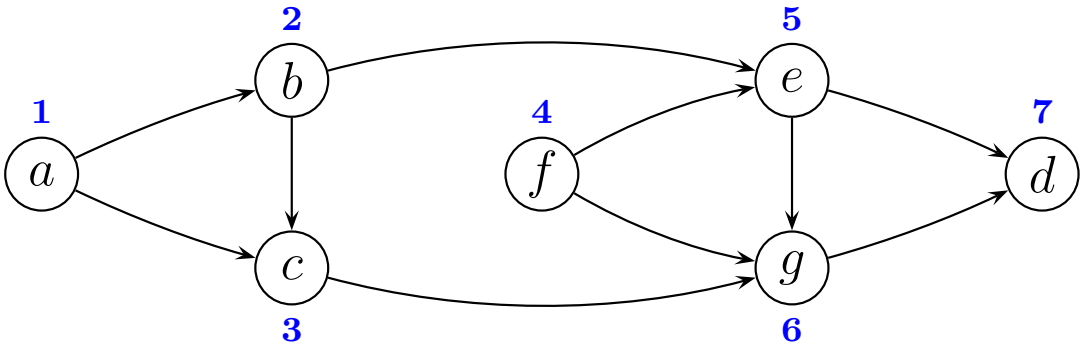
Ce diagramme permet de visualiser la structure d'un DAG. Tous les arcs sont orientés dans une même direction dévoilant ainsi un *flux* allant des sources vers les puits.

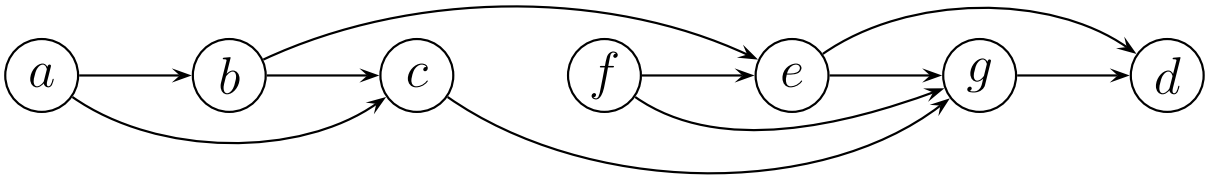
Applications

Les DAG permettent de représenter des situations où la notion même de circuit est erronée (réseaux de distribution d'électricité, réseaux de distribution d'eau, etc). Un problème associé est celui de l'optimisation d'un flux de circulation.

Les DAG permettent aussi de modéliser des problèmes d'ordonnancement.

Illustration





tous les arcs sont orientés de la gauche vers la droite

4. NOTION D'ORDONNANCEMENT

Le problème

Considérons un projet décomposé en tâches soumises à des contraintes de précédence : une tâche x doit être terminée avant qu'une tâche y ne commence. De plus, à chaque tâche correspond une durée d'exécution. Le problème est alors de déterminer un ordre d'exécution des tâches ou ordonnancement des tâches qui minimise la durée totale d'exécution du projet.

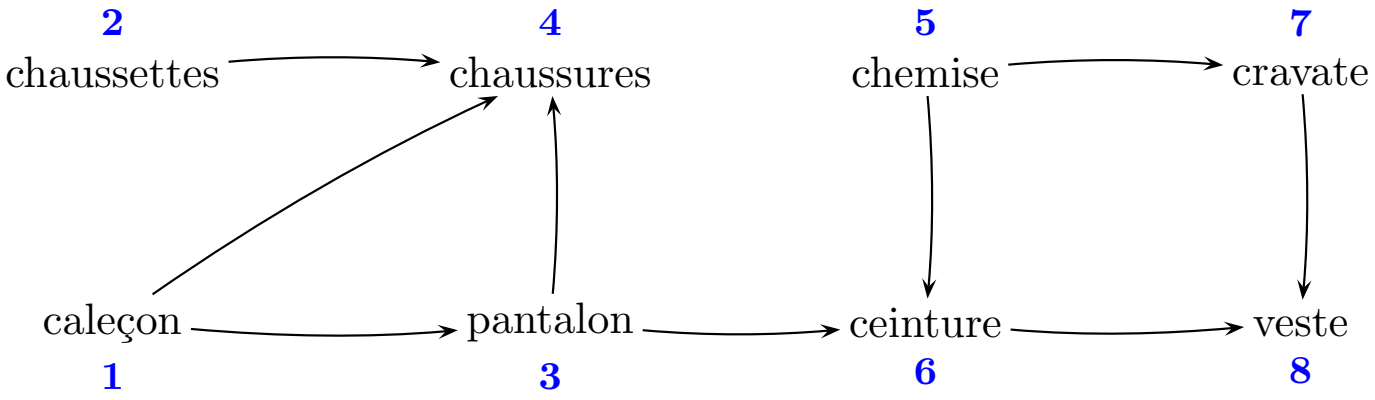
Première modélisation (méthode MPM)

On construit le DAG suivant : les sommets représentent les tâches et les arcs représentent les contraintes. Un ordre topologique pour ce DAG est un ordre d'exécution des tâches qui respecte les contraintes de précédence.

Seconde modélisation (méthode PERT)

On construit le DAG suivant : les arcs représentent les tâches et les sommets représentent des étapes dans l'exécution du projet. Les arcs d'origine un sommet s peuvent commencer quand toutes les tâches de but s sont terminées.

Illustration



un ordre topologique possible est : (caleçon, chaussettes, pantalon, chaussures, chemise, ceinture, cravate, veste).

Propriété 5.1

L'algorithme de décision (§2) supprime les sommets dans un ordre topologique pour le graphe traité.

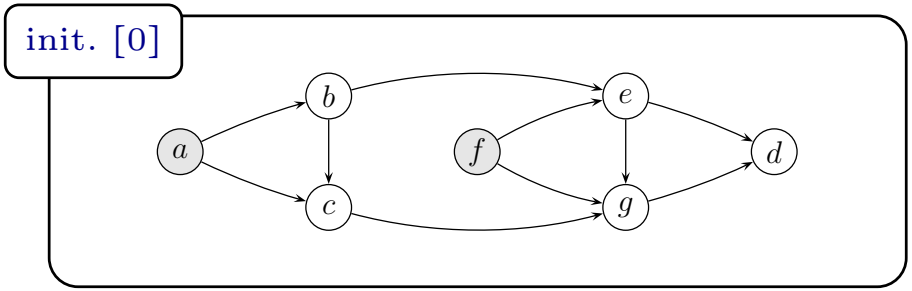
Soit G un DAG d'ordre n et soit $L = (s_1, s_2, \ldots, s_n)$ la liste ordonnée des sommets de G supprimés par l'algorithme de décision. Le sommet s_1 est une source de G : $\varphi(s_1) \leftarrow 1$. Le k^{e} sommet supprimé s_k, $1 < k \leq n$, est tel que $V^-(s_k) \subseteq \{s_1, \ldots, s_{k-1}\}$ et tout arc de la forme (x, s_k) vérifie $\varphi(x) < \varphi(s_k)$.

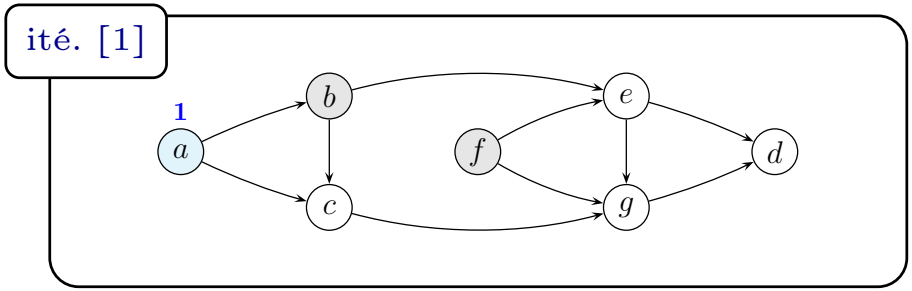
Algorithme de base

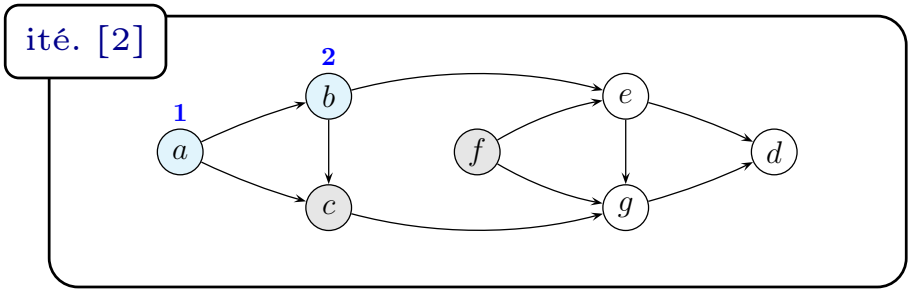
On complète l'algorithme de décision en numérotant les sommets dans l'ordre de suppression. Pour illustrer son fonctionnement, on le déroule sur le diagramme du graphe :

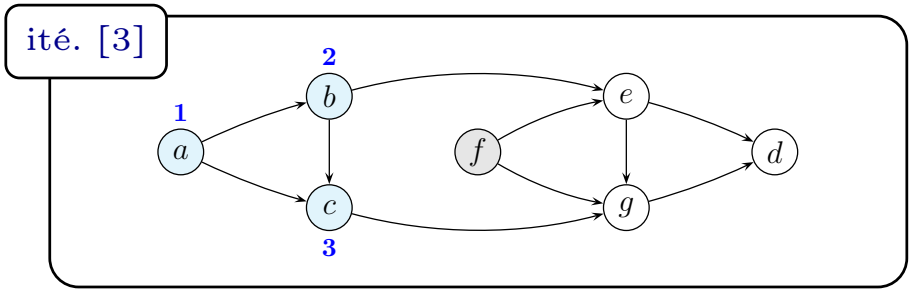
- on simule la suppression d'un candidat s, on notant le numéro $\varphi(s)$ à côté de s sur le diagramme,
- un sommet devient candidat quand tous ses prédécesseurs sont numérotés.

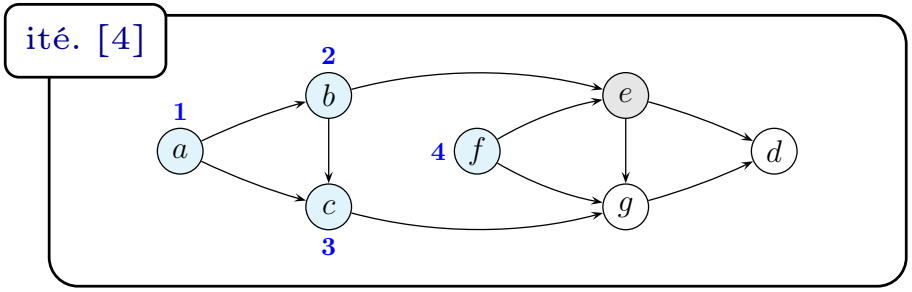
Illustration

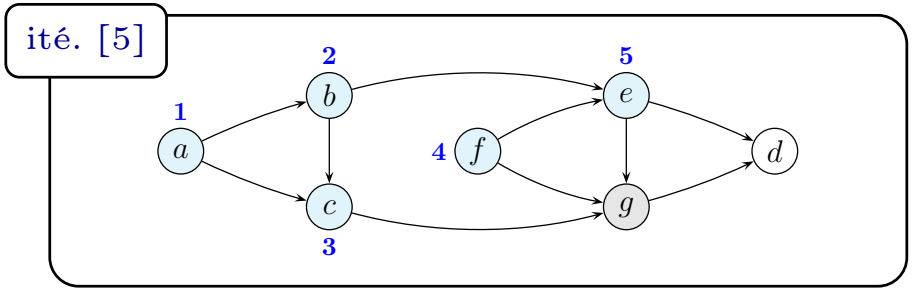


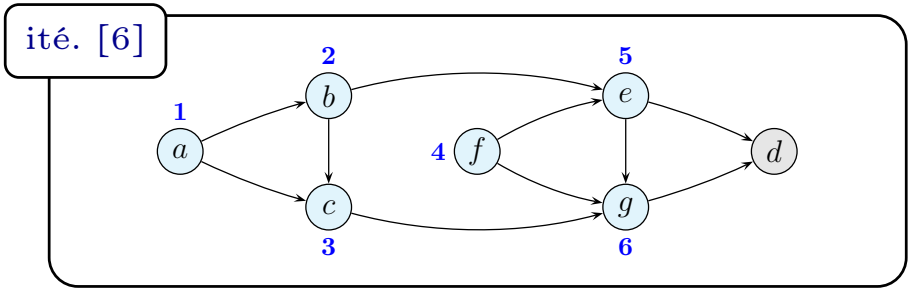


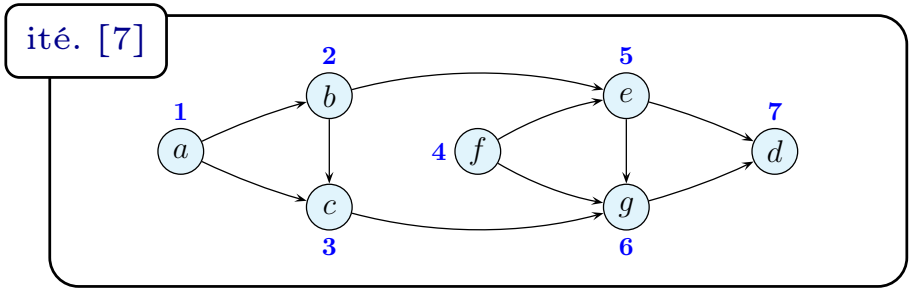












Implémentation : gestion des candidats

- Mémorisation des candidats : utilisation d'un ensemble C.
- Identification des nouveaux candidats : utilisation d'un **marquage des sommets**. Marquer un sommet c'est associer une information à ce sommet. Cette information/marque doit permettre de détecter le moment où un sommet devient candidat. On note $\boldsymbol{marque[s]}$ la marque du sommet s.

Remarque

Pour mémoriser les candidats, on utilise le plus souvent une structure de file (FIFO : First In First Out) au lieu d'une structure d'ensemble.

Implémentation : initialisations

Donnée : $G = (S, A)$, un graphe orienté
Résultat : φ, un ordre topologique pour G si G est un DAG

$C \leftarrow \varnothing$ ..... ✎ l'ensemble des candidats
$k \leftarrow 0$ ..... ✎ k : un compteur (numéro topologique)
pour tout $s \in S$ **faire**
  $\varphi(s) \leftarrow 0$
  $marque[s] \leftarrow \delta^-(s)$ ..... ✎ la marque d'un sommet est son degré intérieur
  si $marque[s] = 0$ **alors** $C \leftarrow C \cup \{s\}$ ..... ✎ ajout des sources dans C
fin pour

Commentaires

La marque d'un sommet est son degré intérieur, c.-à-d. son nombre de prédécesseurs. Un sommet de marque 0 est une source et il est ajouté à l'ensemble des candidats C.

Implémentation : le traitement

tant que $C \neq \varnothing$ **faire**
  $s \leftarrow \text{choisir}(C)$ ..... ✎ choisir un candidat et l'extraire de C
  $\varphi(s) \leftarrow k \leftarrow k+1$ ..... ✎ numérotation du sommet s
  examiner(s) ..... ✎ un traitement éventuel du sommet s
  pour tout $x \in V^+(s)$ **faire**
    $marque[x] \leftarrow marque[x] - 1$
    si $marque[x] = 0$ **alors** $C \leftarrow C \cup \{s\}$ ..... ✎ x devient candidat
  fin pour
fin tq

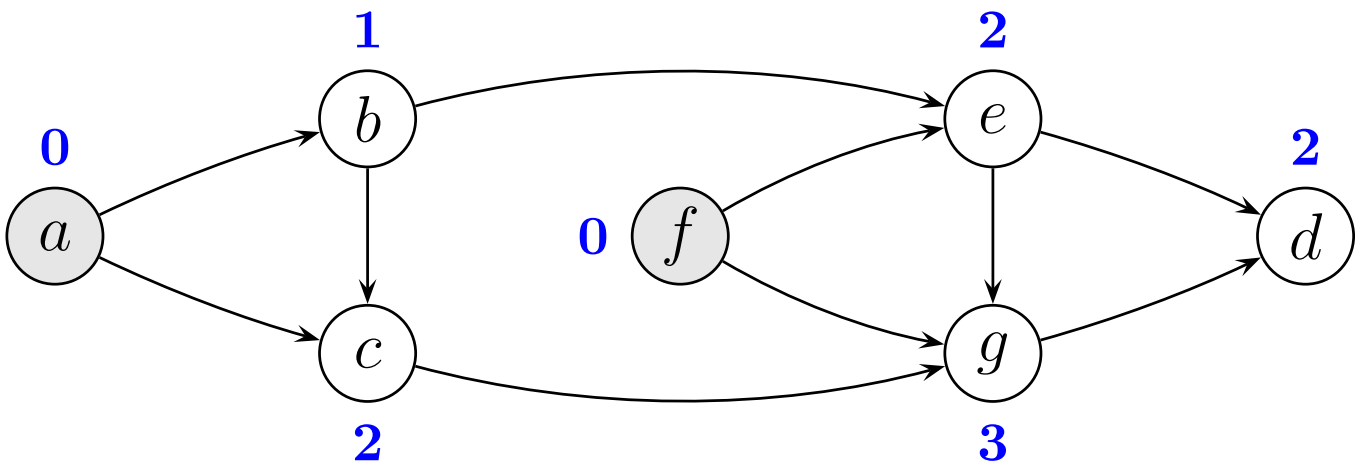
Commentaires

À chaque itération de la boucle **tant que**, on choisit un candidat s et on le numérote. Pour simuler sa suppression, on décrémente la marque des successeurs de s (ils ont un prédécesseur de moins dans le graphe traité). Ceux dont la marque s'annule deviennent à leur tour candidat. Si G est un DAG, l'algorithme fournit un ordre topologique en $n = |S|$ itérations. Sinon, φ ne sera pas entièrement calculée.

Trace de l'algorithme

ité. [0]

initialisation des marques
et de l'ensemble des candidats C

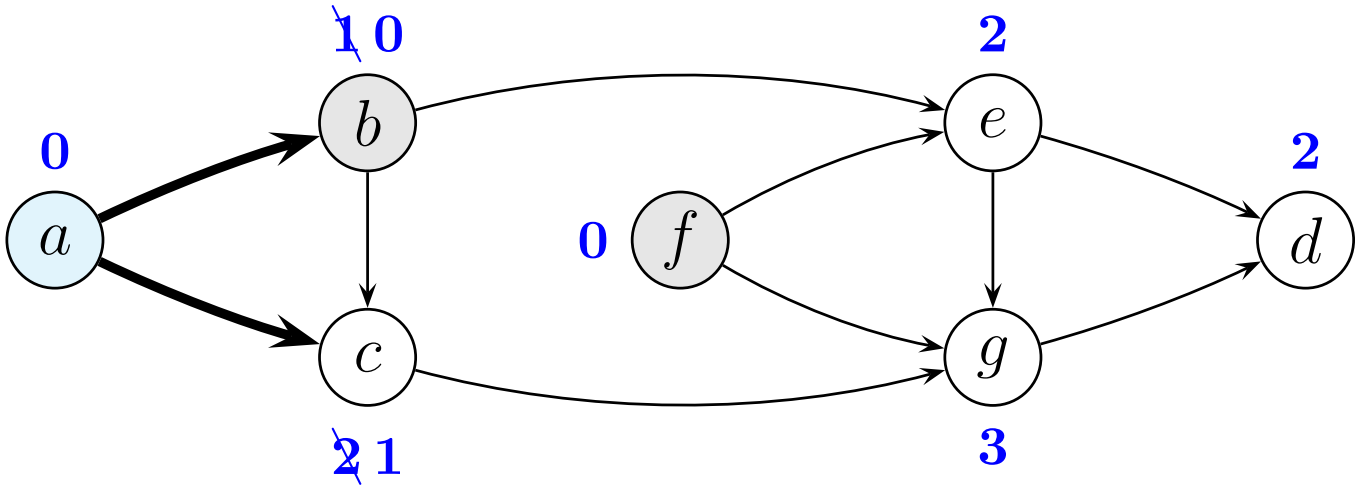


C	a, f
s	
$\varphi(s)$	

Trace de l'algorithme

ité. [1]

numérotation de a, décrémentation
des marques des successeurs de a et maj de C

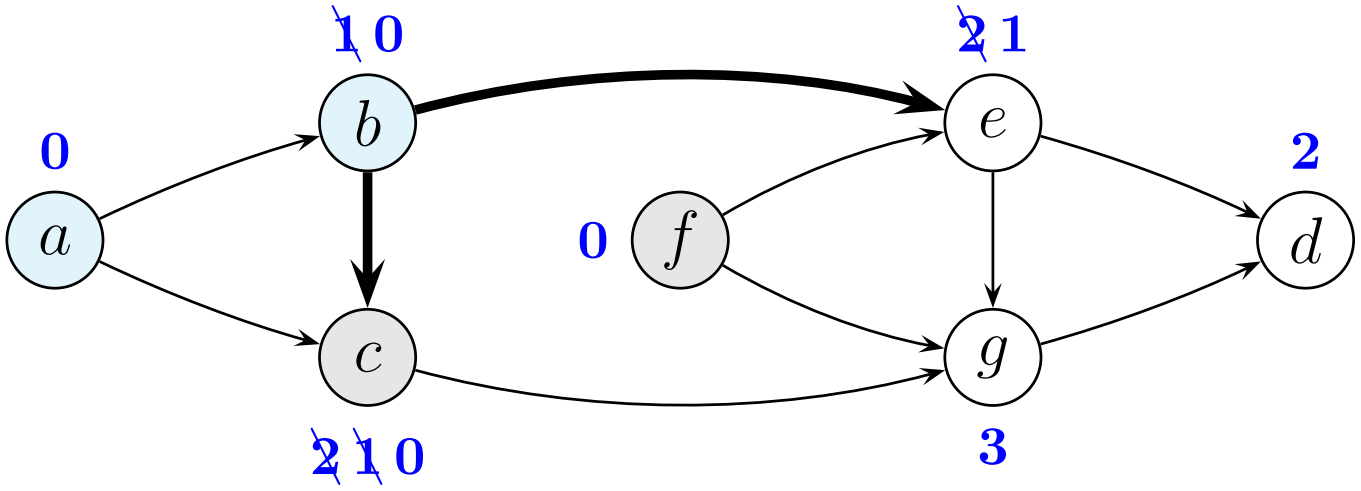


C	a, f	b, f
s	a	
$\varphi(s)$	1	

Trace de l'algorithme

ité. [2]

numérotation de b, décrémentation
des marques des successeurs de b et maj de C

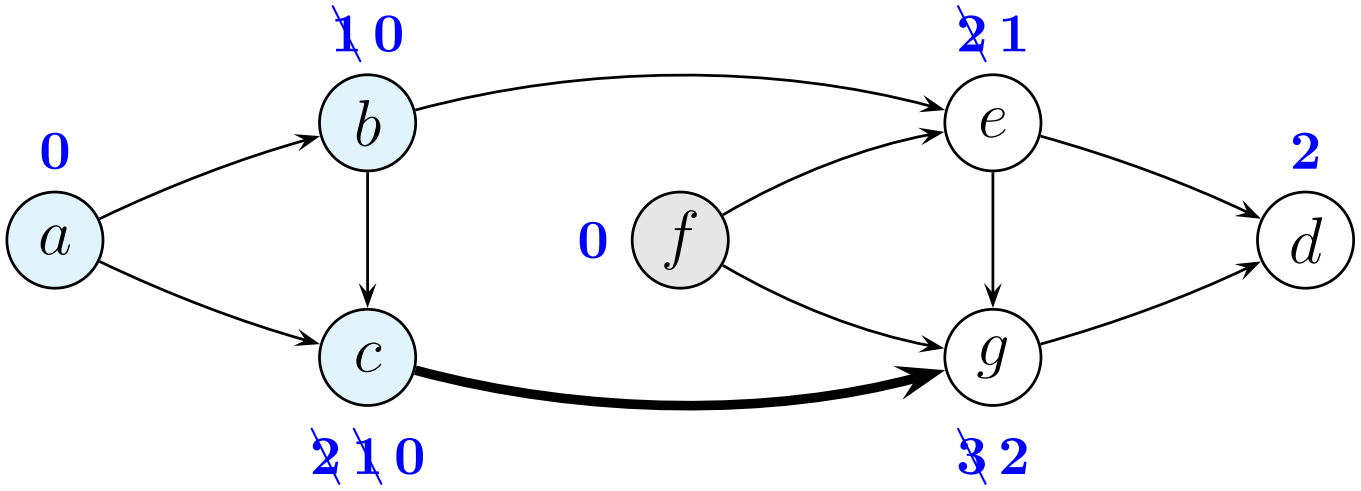


C	a, f	b, f	c, f
s	a	b	
$\varphi(s)$	1	2	

Trace de l'algorithme

ité. [3]

numérotation de c, décrémentation
des marques des successeurs de c et maj de C

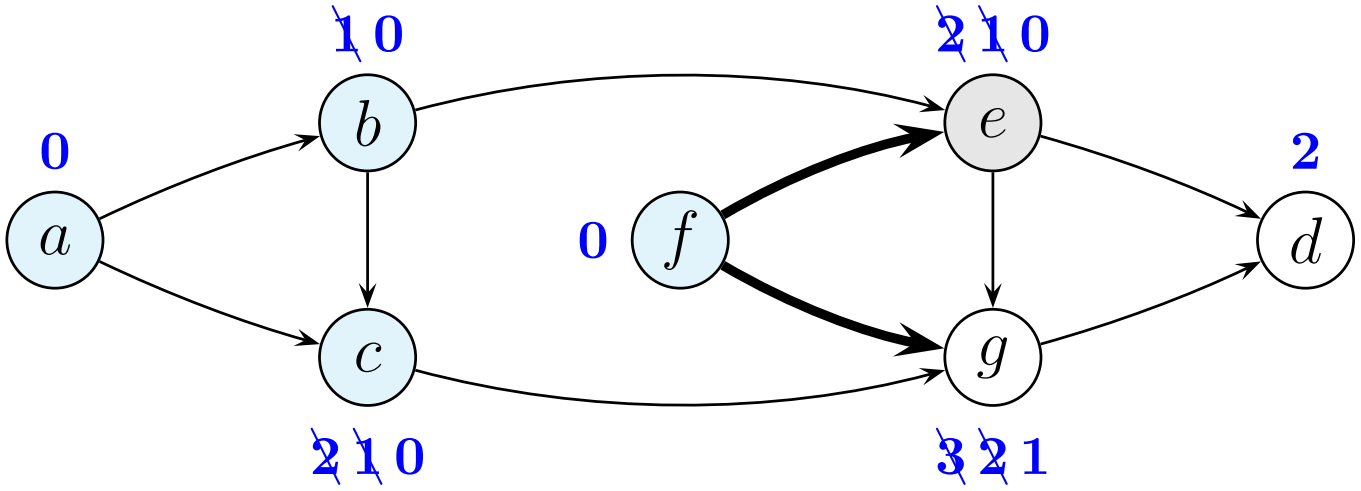


C	a, f	b, f	c, f	f
s	a	b	c	
$\varphi(s)$	1	2	3	

Trace de l'algorithme

ité. [4]

numérotation de f, décrémentation
des marques des successeurs de f et maj de C

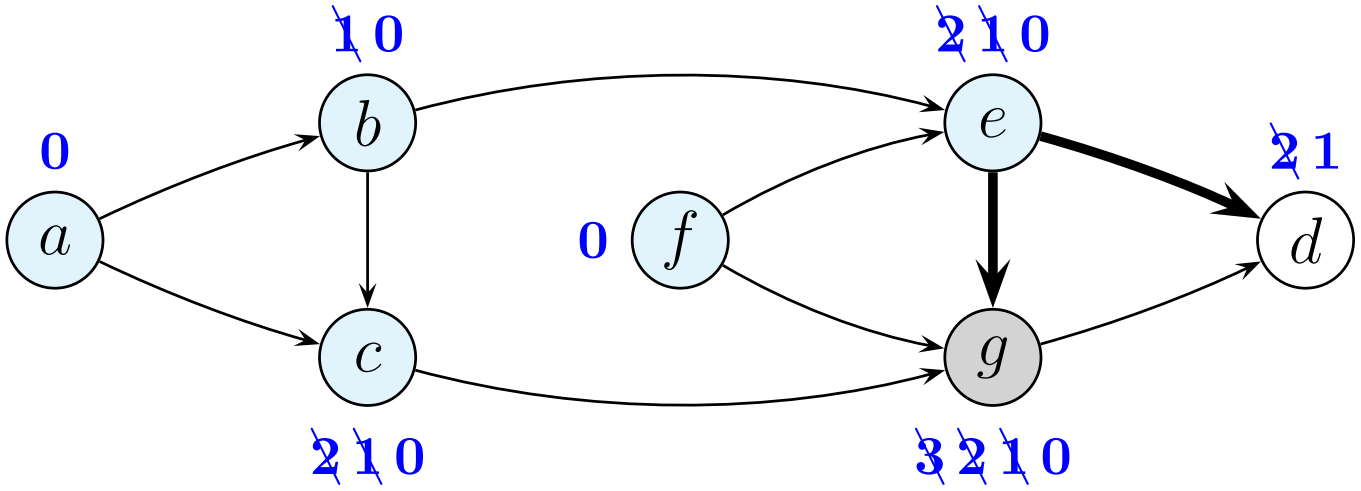


C	a, f	b, f	c, f	f	e
s	a	b	c	f	
$\varphi(s)$	1	2	3	4	

Trace de l'algorithme

ité. [5]

numérotation de e, décrémentation
des marques des successeurs de e et maj de C

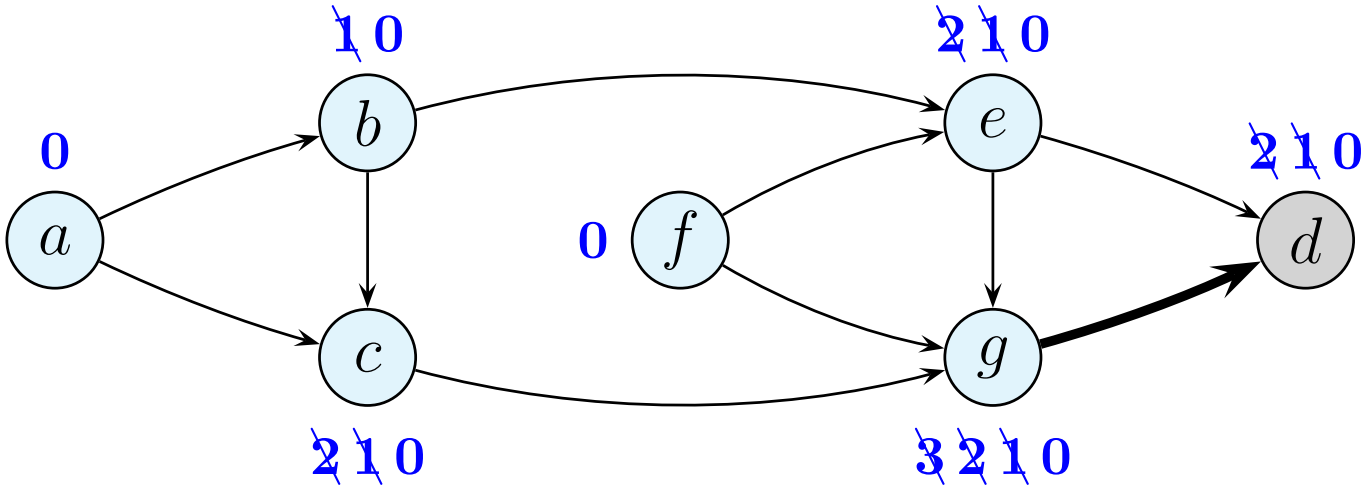


C	a, f	b, f	c, f	f	e	g
s	a	b	c	f	e	
$\varphi(s)$	1	2	3	4	5	

Trace de l'algorithme

ité. [6]

numérotation de g, décrémentation
des marques des successeurs de g et maj de C



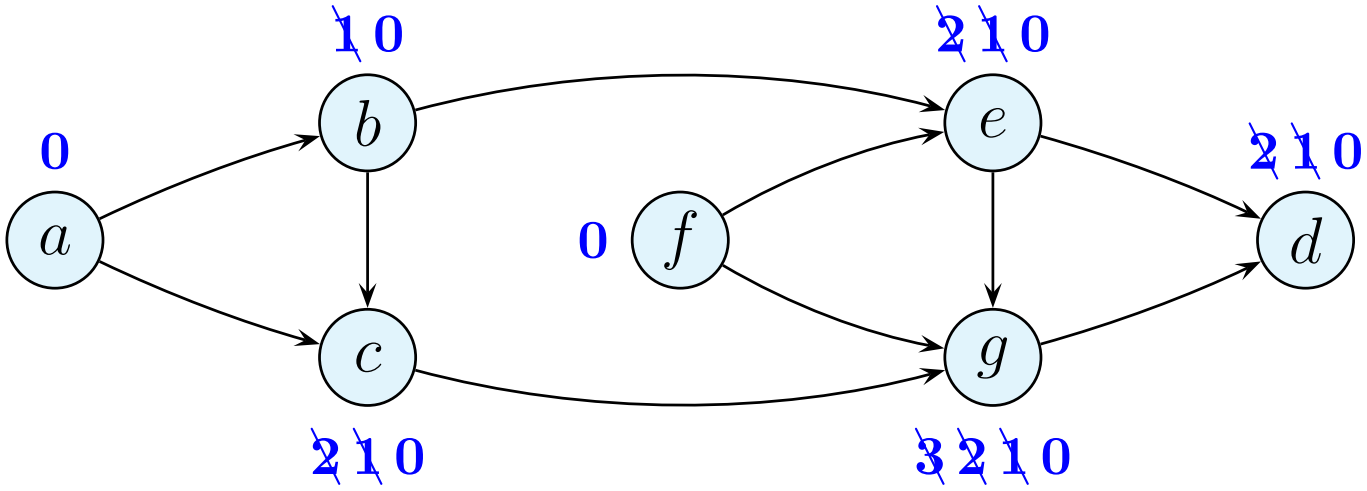
C	a, f	b, f	c, f	f	e	g	d
s	a	b	c	f	e	g	
$\varphi(s)$	1	2	3	4	5	6	

Trace de l'algorithme

ité. [7]

numérotation de d, C est vide

l'algorithme a déterminé un ordre topologique pour le graphe



C	a, f	b, f	c, f	f	e	g	d
s	a	b	c	f	e	g	d
$\varphi(s)$	1	2	3	4	5	6	7

Un algorithme générique

Parmi les problèmes pouvant être modélisés par des graphes, certains peuvent être résolus par un algorithme générique du type suivant :

pour tout $s \in S$ **faire**
| examiner(s) . . . . . ✎ un traitement appliqué au sommet s
fin pour

Commentaires

La boucle **pour** ci-dessus n'indique pas l'ordre dans lequel les sommets doivent être examinés. Cela signifie que, peu importe cet ordre, le résultat obtenu sera une solution du problème.

D'autres problème nécessitent d'examiner les sommets dans un ordre bien précis. C'est le cas, par exemple, de l'algorithme de Welsh et Powell qui détermine une coloration d'un graphe.

Algorithme générique et ordre topologique

Si on examine les sommets d'un DAG dans un ordre topologique alors tout sommet sera examiné **après ses prédécesseurs** (et avant ses successeurs). Au moment d'examiner un sommet, on disposera alors des résultats obtenus lors du traitement de ses prédécesseurs (et de tous ses antécédents).

Il est aussi possible d'examiner les sommets dans un ordre topologique inverse (le premier sommet examiné est un puits). Si on examine les sommets dans cet ordre, tout sommet sera alors examiné **après ses successeurs** (et plus généralement après ses descendants).

Exemple

La méthode PERT examine les sommets dans un ordre topologique pour déterminer les dates de début au plus tôt. Elle examine ensuite les sommets dans un ordre topologique inverse pour déterminer les dates de début au plus tard.